OS ORANGE PROBLEM

Name: Deeptha S
SRN: PES1UG24CS144
Section: 4 'C'

Phase 1: Object Storage Foundation

1a.

```
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$
```

1b.

```
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$
```

Phase 2: Tree Objects

2a.

```
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ make cle
an
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test
_objects.o test_tree.o
rm -rf .pes
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ make tes
t_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ ./test_t
ree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$
```

2b.

```
shankar@Pavilion16:~/PES1UG24CS144-pes-vcs$ find .pes/objects -type f
.pes/objects/02/1657cde88428337031b3a946b7d1253ec80a7893aaa22dc4c2332ac201b2
d3
shankar@Pavilion16:~/PES1UG24CS144-pes-vcs$ xxd .pes/objects/02/1657cde88428
337031b3a946b7d1253ec80a7893aaa22dc4c2332ac201b2d3 | head -20
00000000: 7472 6565 2031 3339 0031 3030 3634 3420  tree 139.100644
00000010: 5245 4144 4d45 2e6d 6400 aaaa aaaa aaaa  README.md.......
00000020: aaaa aaaa aaaa aaaa aaaa aaaa aaaa aaaa  ................
00000030: aaaa aaaa aaaa aaaa aaaa 3130 3037 3535  ..........100755
00000040: 2062 7569 6c64 2e73 6800 cccc cccc cccc   build.sh.......
00000050: cccc cccc cccc cccc cccc cccc cccc cccc  ................
00000060: cccc cccc cccc cccc cccc 3430 3030 3020  ..........40000
00000070: 7372 6300 bbbb bbbb bbbb bbbb bbbb bbbb  src.............
00000080: bbbb bbbb bbbb bbbb bbbb bbbb bbbb bbbb  ................
00000090: bbbb bbbb                                ....
shankar@Pavilion16:~/PES1UG24CS144-pes-vcs$
```

Phase 3: The Index (Staging Area)

3a.

```
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ make pes
rm -rf .pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

3b.

```
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ cat .pes/index
100755 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776666993 6 file1.txt
100755 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776666993 6 file2.txt
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$
```

Phase 4: Commits and History

4a.

```
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ cd /mnt/c/Users/shankar/OneDrive/Desktop/OS_orange
make clean
make pes

rm -rf .pes
./pes init

echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Committed: 91dccee13184... Initial commit
Committed: 2cdc0cb463d9... Add world
Committed: 060aa66555f9... Add farewell
commit 060aa66555f973a40a9ddf324064e67d419dbd71c91cf339095d4198705ea202
Author: Deeptha S <PES1UG24CS144>
Date:   1776667706

    Add farewell

commit 2cdc0cb463d949165ca774e98f8cf9f81ad372ed00878b43f97c3c7f4d206d19
Author: Deeptha S <PES1UG24CS144>
Date:   1776667705

    Add world

commit 91dccee1318478cf7aa30fd3d7d9f86b60131b68ffa44c103c7a8c181ced8e74
Author: Deeptha S <PES1UG24CS144>
Date:   1776667705

    Initial commit
```

4b.

```
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/06/0aa66555f973a40a9ddf324064e67d419dbd71c91cf339095d4198705ea202
.pes/objects/2c/dc0cb463d949165ca774e98f8cf9f81ad372ed00878b43f97c3c7f4d206d19
.pes/objects/55/8c1061e21bccbeb23c527d8dcef5c8646cabfb6091aae3ae7ee3320c9ff7b7
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/91/dccee1318478cf7aa30fd3d7d9f86b60131b68ffa44c103c7a8c181ced8e74
.pes/objects/cc/e49eb2943f54e8135f291862bdf218064409ba05a7ee7d5bec00ddc22addfb
.pes/objects/dd/82340b5e403e2dc18bfca050e6699935849710b2b5d28ab8bbffbb72d32627
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$
```

4c.

```
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ cat .pes/refs/heads/main
060aa66555f973a40a9ddf324064e67d419dbd71c91cf339095d4198705ea202
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$ cat .pes/HEAD
ref: refs/heads/main
shankar@Pavilion16:/mnt/c/Users/shankar/OneDrive/Desktop/OS_orange$
```

Phase 5 & 6: Analysis-Only Questions

Q5.1 — Branching and Checkout

A branch in Git is represented as a file in .git/refs/heads/ that stores the latest commit hash of that branch. To implement pes checkout <branch>, the system must update .pes/HEAD to point to the selected branch (e.g., ref: refs/heads/<branch>). Then, it must read the commit hash from .pes/refs/heads/<branch>, retrieve the corresponding tree object, and update the working directory to match that snapshot. This involves deleting files not present in the target tree and restoring files that exist in it. The operation is complex because it must carefully synchronize the working directory, index, and object store without losing user data. Handling untracked files, nested directories, and partially modified files adds further complexity.

Q5.2 — Dirty Working Directory Detection

To detect conflicts during checkout, the system must compare the working directory with the index and the target branch. First, for each tracked file in the index, compute its current hash from the working directory and compare it with the stored hash in the index. If they differ, the file is considered modified (dirty). Next, compare the index version of the file with the version in the target branch's tree. If the file differs between branches and is also modified locally, checkout must refuse to proceed to avoid overwriting changes. This approach uses only the index (for current tracked state) and object store (for committed versions) to safely detect conflicts.

Q5.3 — Detached HEAD

A detached HEAD occurs when .pes/HEAD contains a direct commit hash instead of a branch reference. In this state, new commits can still be created, but they are not associated with any branch. As a result, these commits can become unreachable if the user switches branches, eventually leading to their deletion during garbage collection. To recover such commits, the user can create a new branch pointing to the current commit (e.g., by writing the hash into .pes/refs/heads/<new-branch>). This reattaches the commits to a branch and prevents them from being lost.

Q6.1 — Garbage Collection Algorithm

Garbage collection identifies and removes unreachable objects from the object store. The algorithm begins by collecting all root references, such as branch heads in .pes/refs/heads/. From each commit, it traverses the commit graph recursively, marking all reachable commits, trees, and blobs. A hash set is used to efficiently track visited objects and avoid duplicate processing. After traversal, the system scans all objects in .pes/objects/ and deletes those not

present in the reachable set. For a repository with 100,000 commits and 50 branches, traversal typically visits each commit once, along with associated trees and blobs, resulting in roughly a few hundred thousand total objects.

Q6.2 — GC Race Condition

Running garbage collection concurrently with a commit operation is dangerous due to race conditions. For example, while a commit is being created, its objects (tree and blob) may be written but not yet referenced by any branch. If GC runs at this moment, it may consider these objects unreachable and delete them. The commit operation would then reference missing objects, corrupting the repository. Git avoids this by using mechanisms such as object locking, temporary files, and atomic renaming. Objects are first written to temporary locations and only made visible after being fully created. Additionally, Git ensures GC does not run concurrently with critical operations, preventing such inconsistencies.